Московский авиационный институт
(национальный исследовательский университет)

Институт №8 «Компьютерные науки и прикладная математика»

Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №6 по курсу «Информационный поиск»

Студент:	Е. Р. Николаева
Преподаватель:	А. А. Кухтичев
Группа:	М8О-401Б-22
Дата:	
Оценка:	
Подпись:	

Москва, 2026

Лабораторная работа №6 «Булев индекс»

Требуется построить поисковый индекс, пригодный для булева поиска, по подготовленному в ЛР1 корпусу документов. Требования к индексу:

- Самостоятельно разработанный, бинарный формат представления данных. Формат необходимо описать в отчёте, в побайтовом (или побитовом) представлении.
- Формат должен предполагать расширение, т.к. в следующих работах он будет меняться под требования новых лабораторных работ.
- Использование текстового представления или готовых баз данных не допускается.
- Кроме обратного индекса, должен быть создан «прямой» индекс, содержащий в себе как минимум заголовки документов и ссылки на них (понадобятся для выполнения ЛР4, при генерации страницы поисковой выдачи).
- Для термов должна быть как минимум понижена капитализация. В отчёте должно быть отмечено как минимум:
- Выбранное внутренне представление документов после токенизации.
- Выбранный метод сортировки, его достоинства и недостатки для задачи индексации. Среди результатов и выводов работы нужно указать:
- Количество термов.
- Средняя длина терма. Сравнить со средней длинной токена, вычисленной в ЛР1 по курсу ОТЕЯ. Объяснить причину отличий.
- Скорость индексации: общую, в расчёте на один документ, на килобайт текста.
- Оптимальна ли работа индексации? Что можно ускорить? Каким образом? Чем она ограниченна? Что произойдёт, если объём входных данных увеличится в 10 раз, в 100 раз, в 1000 раз?

1 Описание работы

В данной лабораторной работе требовалось построить поисковый индекс, пригодный для булева поиска, по подготовленному ранее корпусу документов. Основным требованием являлось использование собственного бинарного формата хранения индекса. Кроме обратного индекса, необходимо было реализовать прямой индекс, содержащий как минимум заголовки документов и ссылки на них. Также требовалось обеспечить возможность дальнейшего расширения формата в следующих лабораторных работах.

2 Постановка задачи

В рамках лабораторной работы необходимо:

- построить обратный индекс по корпусу документов;
- реализовать прямой индекс;
- использовать собственный бинарный формат представления данных;
- отказаться от текстового хранения индекса и готовых баз данных;
- обеспечить расширяемость формата;
- понизить капитализацию термов;
- вычислить статистические и временные характеристики индексации.

3 Общее устройство индекса

В реализованной системе индекс хранится в виде набора бинарных файлов:

- `index_header.bin`;
- `forward.bin`;
- `vocab.bin`;
- `postings.bin`.

Такое разделение было выбрано по следующим причинам:

- упрощается организация данных;

- словарь и postings-списки могут храниться отдельно;
- прямой индекс удобно выделить в самостоятельный файл;
- формат становится проще расширять в будущих лабораторных работах.

4 Внутреннее представление документа после токенизации

После токенизации каждый документ рассматривается как последовательность термов, приведённых к нижнему регистру. На следующем этапе для документа вычисляется отображение вида:

$$\texttt{term} \to \texttt{tf},$$

где `tf` — число вхождений данного терма в документ.

Таким образом, внутренне документ после токенизации представляется двумя связанными способами:

- как последовательность токенов;
- как таблица частот термов документа.

При построении индекса для каждого уникального терма документа формируется posting, содержащий идентификатор документа и частоту терма в документе.

5 Нормализация термов

В соответствии с требованиями задания для всех термов была понижена капитализация. Это достигается на этапе токенизации переводом символов к нижнему регистру. Такой подход позволяет считать одинаковыми, например, формы `Model`, `MODEL` и `model`, что уменьшает размер словаря и делает поиск более устойчивым.

6 Структура бинарного формата

1 Файл `index_header.bin`

Файл заголовка содержит основную служебную информацию об индексе. В реализации использовалась следующая структура:

- `magic[8]` — сигнатура формата;
- `version (uint32)`;
- `doc_count (uint32)`;
- `term_count (uint32)`;
- `reserved (uint32)`.

Побайтово структура заголовка имеет вид:

Смещение	Размер	Содержимое
0	8 байт	magic
8	4 байта	version
12	4 байта	doc_count
16	4 байта	term_count
20	4 байта	reserved

Наличие полей `version` и `reserved` обеспечивает расширяемость формата.

2 Файл `forward.bin`

Прямой индекс содержит информацию о документах. Для каждого документа сохраняется запись следующего вида:

- `doc_id (uint32)`;
- `title_len (uint32)`;
- `path_len (uint32)`;
- `token_count (uint32)`;
- байты заголовка;
- байты пути к документу.

Таким образом, запись прямого индекса имеет переменную длину. Это позволяет хранить заголовки и ссылки без заранее фиксированного размера поля.

3 Файл `vocab.bin`

Файл словаря содержит записи для каждого терма. Для одного терма сохраняются:

- `term_len (uint32)`;
- байты терма;
- `df (uint32)`;
- `postings_offset (uint64)`;
- `postings_count (uint32)`;
- `reserved (uint32)`.

Поле `df` содержит document frequency, то есть число документов, в которых встретился терм. Поле `postings_offset` указывает смещение в файле `postings.bin`, с которого начинается список postings для данного терма.

4 Файл `postings.bin`

Файл postings-списков хранит пары:

- `doc_id (uint32)`;
- `tf (uint32)`.

Хотя для чисто булева поиска достаточно было бы хранить только `doc_id`, в реализации сразу сохранялась и частота терма в документе. Это делает формат более универсальным и позволяет использовать его в следующих лабораторных работах.

7 Обоснование расширяемости формата

Формат изначально проектировался как расширяемый. Это проявляется в следующем:

- в заголовке есть поле версии;
- в записях предусмотрены резервные поля;
- словарь, postings и прямой индекс хранятся раздельно;

- структура файла допускает добавление новых атрибутов документа и терма без полного изменения общей архитектуры.

Такой подход удобен, поскольку в следующих лабораторных работах к индексу могут добавляться дополнительные поля, новые типы статистики, позиции термов, веса и другая служебная информация.

8 Метод сортировки

После накопления словаря и postings-списков термы сортировались лексикографически с помощью внутренней сортировки в памяти (`std::sort`).

Использованный подход имеет следующие достоинства:

- простота реализации;
- высокая скорость на объёмах данных, помещающихся в оперативную память;
- удобство формирования упорядоченного словаря.

Недостатки данного метода:

- зависимость от доступного объёма памяти;
- при росте объёма коллекции может потребоваться внешняя сортировка;
- in-memory сортировка плохо масштабируется на очень большие коллекции.

В рамках текущего корпуса выбранный метод является оправданным, так как объём данных ещё позволяет работать полностью в памяти.

9 Реализация

Индексатор был реализован на языке C++. После чтения текста документа выполняется токенизация, затем подсчёт частот термов в документе, после чего эти данные встраиваются в глобальный индекс.

Подсчёт частот термов внутри документа реализуется следующим образом:

```
unordered_map<string,uint32_t>tf;
for (const string& tok : tokens) {
tf[tok]++;
}
```

После этого для каждого терма создаётся posting:

```
for (const auto& kv : tf) {
index[kv.first].push_back({doc_id,kv.second});
}
```

Для записи бинарных файлов использовалась вспомогательная функция:

```
template <typename T>
void write_binary(ofstream& out,const T& value) {
out.write(reinterpret_cast<const char*>(&value),sizeof(T));
}
```

С её помощью последовательно записывались целочисленные поля заголовка, прямого индекса, словаря и postings-списков.

10 Прямой индекс

Для каждого документа сохранялись:

- идентификатор документа;
- заголовок;
- путь к файлу;
- число токенов в документе.

Заголовок извлекался из текста документа с помощью отдельной функции. Прямой индекс нужен не только для текущей лабораторной работы, но и для последующих, так как именно он используется при формировании поисковой выдачи.

11 Результаты выполнения

В результате выполнения лабораторной работы был построен бинарный индекс, пригодный для булева поиска. Индекс содержит:

- прямой индекс по документам;
- словарь термов;

- postings-списки;
- служебный заголовок индекса.

В ходе индексации были вычислены следующие характеристики:

- количество термов;
- средняя длина терма;
- общее время индексации;
- скорость индексации в пересчёте на документ;
- скорость индексации в пересчёте на килобайт текста.

- Terms count: 1030452
- Average term length: 9.37078
- Input size (bytes): 1201818038
- Execution time (sec): 87.2703
- Indexing speed (docs/sec): 458.346
- Time per document (sec/doc): 0.00218176

Эти значения выводились программой после завершения построения индекса. Полученные результаты показывают, что индексатор способен обработать корпус из 40000 документов за приемлемое время. При этом словарь термов получился достаточно большим, что естественно для тематического научного корпуса и связано с большим числом редких и специализированных терминов.

12 Средняя длина терма и отличие от средней длины токена

Средняя длина терма в построенном индексе составила 9.37078, тогда как в лабораторной работе по токенизации средняя длина токена составляла 4.930926.

Такое различие объясняется способом подсчёта. В ЛР3 средняя длина вычислялась по всем токенам корпуса с учётом повторов. Это означает, что очень частые короткие слова, такие как служебные элементы языка, сильно влияют на итоговое среднее значение и уменьшают его.

В данной лабораторной работе средняя длина вычисляется по уникальным термам словаря. Каждый терм учитывается только один раз, поэтому редкие длинные научные слова, аббревиатуры, составные термины и специализированные обозначения оказывают на итоговое среднее значительно большее влияние. По этой причине средняя длина терма словаря заметно выше средней длины токена по корпусу.

Частые короткие служебные слова вроде `the`, `of`, `and` сильно влияют на среднюю длину токена, так как встречаются огромное число раз. В словаре же каждый терм учитывается только один раз, поэтому редкие длинные научные термины повышают среднюю длину терма. По этой причине средняя длина терма обычно оказывается больше средней длины токена.

13 Анализ производительности

Время индексации определяется в основном следующими факторами:

- скоростью чтения документов с диска;
- токенизацией текста;
- построением структуры частот термов;
- накоплением postings-списков;
- сортировкой словаря;
- записью бинарных файлов.

Текущую реализацию нельзя считать строго оптимальной, поскольку она строит весь индекс в памяти. Такой подход удобен и быстр на умеренном объёме данных, но плохо масштабируется при очень большом размере коллекции.

14 Что можно ускорить

Производительность индексации можно улучшить следующими способами:

- использовать блочную индексацию вместо хранения всего индекса в памяти;
- применять внешнюю сортировку;
- уменьшить число временных строк и копирований;
- использовать более эффективные структуры хранения термов;
- выполнять обработку документов параллельно.

15 Масштабируемость

Если объём входных данных увеличить в 10 раз, текущая реализация, вероятно, ещё сможет работать, но потребует больше оперативной памяти и времени.

При увеличении объёма в 100 раз программа, скорее всего, упрётся в ограничение по памяти, так как словарь и postings-списки станут слишком большими для полностью in-memory индексации.

При увеличении объёма в 1000 раз наивный подход станет практически неприменимым. В этом случае потребуется переход к методам внешней индексации, например SPIMI или BSBI, а также использование сжатия postings-списков.

16 Выводы

В результате выполнения лабораторной работы был реализован бинарный поисковый индекс, пригодный для выполнения булева поиска. Индекс включает прямой и обратный компоненты, хранится в собственном формате и не использует текстовое представление или готовые базы данных. В работе было выбрано внутреннее представление документа через последовательность токенов и таблицу частот термов. Для построения словаря использовалась внутренняя сортировка в памяти, что оказалось удобным и эффективным для текущего объёма корпуса. В то же время анализ показал, что такой подход ограничен объёмом оперативной памяти и при значительном росте данных потребуется переход к внешней индексации. Разработанный формат является достаточно простым, но в то же время допускает расширение. Это делает его пригодным в качестве основы для следующих лабораторных работ, связанных с реализацией поиска и ранжирования.

Список литературы

[1] Маннинг К., Рагхаван П., Шютце Х. *Введение в информационный поиск.* — М.: Вильямс, 2011. — 528 с.

[2] Уиттен И., Моффат А., Белл Т. *Управление гигабайтами: сжатие и индексирование документов и изображений.* — Электронный ресурс.

[3] Джурафски Д., Мартин Д. Х. *Обработка речи и языка.* — Электронный ресурс.

[4] Бэ За-Юн, Рибейро-Нето Б. *Современное информационное извлечение.* — Электронный ресурс.